

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ
И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ
ТЕХНОЛОГИЙ**

**Отчет по лабораторной работе №5
по курсу «Проектирование и реализация баз данных»
Тема: «Процедуры, функции, триггеры в PostgreSQL»**

Выполнила:
Блинова Полина Вячеславовна K3239

Проверила:
Говорова Марина Михайловна

Санкт-Петербург
2026 г.

Оглавление

1 Цель работы	3
2 Практическое задание (вариант 2.2 – авторские триггеры)	4
3 Выполнение работы	5
3.1 Создание процедур	6
Процедура 1. Получение расписания занятий для групп на определенный день недели	6
Процедура 2. Запись на курс слушателя	7
Процедура 3. Получение перечня свободных лекционных аудиторий на любой день недели	9
3.2 Триггеры по ИЗ	11
Триггер 1. Проверка корректности времени занятия (с 8:00 до 22:00)	11
Триггер 2. Автоматический расчет итоговой оценки при выставлении баллов	11
Триггер 3. Защита от изменения расписания после начала занятия	12
Триггер 4. Защита от переполнения аудитории	13
Триггер 5. Запрет на выставление оценки без отметки о посещении	14
Триггер 6. Оценка только после занятия	15
Триггер 7. Автоматическое обновление статуса студента (активен/выпущен)	16
3.3 Модификация триггерной функции Части 2 практического занятия	18

1 Цель работы

Овладеть практическими навыками создания и использования хранимых процедур, функций и триггеров в СУБД PostgreSQL для автоматизации бизнес-логики и обеспечения целостности данных.

Оборудование: компьютерный класс.

ПО: СУБД PostgreSQL, SQL Shell (psql).

2 Практическое задание (вариант 2.2 – авторские триггеры)

Задание содержит следующие пункты:

- Создать 3 хранимые процедуры для БД «Курсы» (вариант 7) с использованием параметров IN/INOUT.
- Создать 3 оригинальных триггера для поддержки бизнес-правил учебного заведения.
- Продемонстрировать работу процедур и триггеров в консоли psql.
- Проверить обработку корректных и некорректных ситуаций.

3 Выполнение работы

Работа выполнялась в СУБД PostgreSQL 18 через консоль psql и pgAdmin.

БД: Courses, схема: courses schema.

Вариант: 7 (по ЛР 2)

```
[polinochka@Polinas-MacBook-Pro ~ % sudo -u postgres /Library/PostgreSQL/18/bin/pg_ctl -D /Library/PostgreSQL/18/data start
shell-init: error retrieving current directory: getcwd: cannot access parent directories: Permission denied
waiting for server to start....shell-init: error retrieving current directory: getcwd: cannot access parent directories: Permission denied
2026-05-17 11:45:18.941 MSK [10022] LOG:  redirecting log output to logging collector process
2026-05-17 11:45:18.941 MSK [10022] HINT:  Future log output will appear in directory "log".
done
server started
[polinochka@Polinas-MacBook-Pro ~ % /Library/PostgreSQL/18/bin/psql -h localhost -p 5433 -U postgres -d Courses ]
[Password for user postgres: ]
psql (18.3)
Type "help" for help.
```

Courses=#

```
Courses=# SET search_path TO courses_schema;
SET
```

```
[Courses=# SELECT current_schema();
current_schema
```

```
-----
courses_schema
(1 row)
```

```
[Courses=# \dt
```

List of tables			
Schema	Name	Type	Owner
courses_schema	assessment	table	postgres
courses_schema	assessment_result	table	postgres
courses_schema	completion_document	table	postgres
courses_schema	contact	table	postgres
courses_schema	education_document	table	postgres
courses_schema	education_program	table	postgres
courses_schema	group_db	table	postgres
courses_schema	group_enrollment	table	postgres
courses_schema	individual	table	postgres
courses_schema	lesson	table	postgres
courses_schema	passport	table	postgres
courses_schema	program_subject	table	postgres
courses_schema	qualification	table	postgres
courses_schema	room	table	postgres
courses_schema	subject	table	postgres
courses_schema	teacher	table	postgres
courses_schema	teacher_competence	table	postgres

(17 rows)

3.1 Создание процедур

Процедура 1. Получение расписания занятий для групп на определенный день недели

Формулировка:

Вывести расписание для указанной группы в заданный день недели (название дисциплины, время начала, номер пары, аудитория, преподаватель).

```
CREATE OR REPLACE PROCEDURE get_schedule(
    p_group_id INT,
    p_day_of_week INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_record RECORD;
    v_target_date DATE;
    v_days_to_add INT;
BEGIN
    v_days_to_add := (p_day_of_week - EXTRACT(DOW FROM
CURRENT_DATE) + 7) % 7;
    v_target_date := CURRENT_DATE + v_days_to_add;

    RAISE NOTICE 'Расписание для группы: %', p_group_id;
    RAISE NOTICE 'Дата: % (%)', v_target_date, CASE WHEN v_days_to_add =
0 THEN 'сегодня' ELSE 'ближайший' END;

    FOR v_record IN
        SELECT DISTINCT
            l.pair_number,
            to_char(l.lesson_datetime, 'HH24:MI') as start_time,
            s.name AS subject,
            r.room_number,
            i.full_name AS teacher
        FROM lesson l
        JOIN subject s ON l.subject_id = s.subject_id
        JOIN room r ON l.room_id = r.room_id
        JOIN teacher t ON l.teacher_id = t.teacher_id
        JOIN individual i ON t.individual_id = i.individual_id
        WHERE l.group_id = p_group_id
```

```

        AND l.lesson_datetime::DATE = v_target_date
    ORDER BY l.pair_number
LOOP
    RAISE NOTICE 'Пара %: % | % | Ауд. % | %',
        v_record.pair_number,
        v_record.start_time,
        v_record.subject,
        v_record.room_number,
        v_record.teacher;
END LOOP;

IF NOT FOUND THEN
    RAISE NOTICE 'Занятий нет';
END IF;
END;
$$;

```

Проверка работы: (1 группа, понедельник)

```

[Courses=# CALL get_schedule(1, 1);
NOTICE: Расписание для группы: 1
NOTICE: Дата: 2026-05-18 (сегодня)
NOTICE: Пара 1: 09:00 | Базы данных | Ауд. А-101 | Кузнецова Алина Сергеевна
NOTICE: Пара 3: 12:20 | Программирование на Python | Ауд. Б-202 | Попова Екатерина Дмитриевна
]

```

(Процедура выводит расписание на ближайшую дату по дню недели)

Процедура 2. Запись на курс слушателя

Формулировка:

Зачислить слушателя (физическое лицо) на указанную образовательную программу. Автоматически подобрать группу (свободное место).

```

CREATE OR REPLACE PROCEDURE enroll_student(
    p_individual_id BIGINT,
    p_group_id BIGINT,
    p_academic_year TEXT,
    p_tuition_fee NUMERIC DEFAULT 0
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_student_count INT;
    v_capacity INT;
BEGIN

```

```

SELECT capacity INTO v_capacity
FROM group_db
WHERE group_id = p_group_id;

SELECT COUNT(*) INTO v_student_count
FROM group_enrollment
WHERE group_id = p_group_id AND academic_year = p_academic_year;

IF v_student_count >= v_capacity THEN
    RAISE EXCEPTION 'Ошибка: в группе % нет свободных мест
(вместимость: %)', p_group_id, v_capacity;
END IF;

INSERT INTO group_enrollment (enrollment_id, individual_id, group_id,
academic_year, tuition_fee)
VALUES (
    (SELECT COALESCE(MAX(enrollment_id), 0) + 1 FROM
group_enrollment),
    p_individual_id,
    p_group_id,
    p_academic_year,
    p_tuition_fee
);

RAISE NOTICE 'Студент % зачислен в группу % на % год (стоимость: %)',
    p_individual_id, p_group_id, p_academic_year, p_tuition_fee;

EXCEPTION
    WHEN unique_violation THEN
        RAISE WARNING 'Студент % уже зачислен в группу % на год %',
            p_individual_id, p_group_id, p_academic_year;
END;
$$;

```

Проверка:

```

[Course=# CALL enroll_student(5, 1, '2026/2027', 120000);
NOTICE: Студент 5 зачислен в группу 1 на 2026/2027 год (стоимость: 120000)
CALL
[Course=# CALL enroll_student(5, 1, '2026/2027', 120000);
WARNING: Студент 5 уже зачислен в группу 1 на год 2026/2027
CALL

```



```

[Courses=# CALL enroll_student(32, 1, '2026/2027', 120000);
ERROR: Ошибка: в группе 1 нет свободных мест (вместимость: 25)
CONTEXT: PL/pgSQL function enroll_student(bigint,bigint,text,numeric) line 15 at RAISE
Courses=# █

```

Процедура 3. Получение перечня свободных лекционных аудиторий на любой день недели

Формулировка:

Вывести список лекционных аудиторий, свободных в указанный день недели. Если свободных нет – выдать соответствующее сообщение.

```

CREATE OR REPLACE PROCEDURE get_free_lecture_rooms(
    p_day_of_week INT,
    p_date DATE DEFAULT CURRENT_DATE
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_room RECORD;
    v_count INT := 0;
BEGIN
    FOR v_room IN
        SELECT r.room_number
        FROM room r
        WHERE r.room_type = 'Лекционная'
        AND NOT EXISTS (
            SELECT 1 FROM lesson l
            WHERE l.room_id = r.room_id
            AND EXTRACT(DOW FROM l.lesson_datetime) = p_day_of_week
            AND DATE(l.lesson_datetime) = p_date
        )
        ORDER BY r.room_number
    LOOP
        RAISE NOTICE 'Аудитория %', v_room.room_number;
        v_count := v_count + 1;
    END LOOP;

    IF v_count = 0 THEN
        RAISE NOTICE 'Свободных лекционных аудиторий нет';
    END IF;

```

END;
\$\$;

Проверка:

```
[Courses=# CALL get_free_lecture_rooms(1, '2026-05-18');  
NOTICE: Аудитория A-202  
CALL  
Courses=# ]
```

Для сравнения:

```
[Courses=# SELECT room_id, room_number FROM room WHERE room_type = 'Лекционная';  
room_id | room_number  
-----+-----  
1 | A-101  
6 | A-202  
(2 rows)  
  
Courses=# SELECT DISTINCT r.room_id, r.room_number  
Courses=# FROM lesson l  
Courses=# JOIN room r ON l.room_id = r.room_id  
Courses=# WHERE r.room_type = 'Лекционная' AND DATE(l.lesson_datetime) = '2026-05-18' AND EXTRACT(DOW FROM l.lesson_datetime) = 1;  
room_id | room_number  
-----+-----  
1 | A-101  
(1 row)]
```

3.2 Триггеры по ИЗ

Триггер 1. Проверка корректности времени занятия (с 8:00 до 22:00)

Назначение: Запретить занятия слишком рано или слишком поздно.

```
CREATE OR REPLACE FUNCTION courses_schema.fn_valid_lesson_time()
RETURNS TRIGGER AS $$
BEGIN
    IF EXTRACT(HOUR FROM NEW.lesson_datetime) < 8
        OR EXTRACT(HOUR FROM NEW.lesson_datetime) >= 22 THEN
        RAISE EXCEPTION 'Занятие может проходить только с 8:00 до 22:00.
Указано время: %',
            NEW.lesson_datetime;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER tr_valid_lesson_time
BEFORE INSERT OR UPDATE ON courses_schema.lesson
FOR EACH ROW
EXECUTE FUNCTION courses_schema.fn_valid_lesson_time();
```

Проверка:

```
[Courses=# INSERT INTO lesson (individual_id, group_id, subject_id, teacher_id, room_id, lesson_datetime,]
pair_number)
Courses=# VALUES (3, 1, 9, 2, 1, '2026-05-20 07:30:00', 1);
ERROR:  Занятие может проходить только с 8:00 до 22:00. Указано время: 2026-05-20 07:30:00+03
CONTEXT:  PL/pgSQL function fn_valid_lesson_time() line 5 at RAISE
Courses=# █
```

Триггер 2. Автоматический расчет итоговой оценки при выставлении баллов

Назначение: при добавлении или изменении балла за занятие автоматически выставлять оценку по 5-балльной шкале.

```
CREATE OR REPLACE FUNCTION auto_calculate_grade()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.grade IS NOT NULL THEN
```

```

IF NEW.grade >= 90 THEN
    NEW.grade_letter := 'Отлично';
ELSIF NEW.grade >= 75 THEN
    NEW.grade_letter := 'Хорошо';
ELSIF NEW.grade >= 60 THEN
    NEW.grade_letter := 'Удовлетворительно';
ELSE
    NEW.grade_letter := 'Неудовлетворительно';
END IF;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

ALTER TABLE lesson ADD COLUMN IF NOT EXISTS grade_letter TEXT;

DROP TRIGGER IF EXISTS tr_auto_calculate_grade ON lesson;
CREATE TRIGGER tr_auto_calculate_grade
    BEFORE INSERT OR UPDATE OF grade ON lesson
    FOR EACH ROW
    EXECUTE FUNCTION auto_calculate_grade();

```

Проверка:

```

Courses=# INSERT INTO lesson (individual_id, group_id, subject_id, teacher_id, room_id, lesson_datetime
, pair_number, semester, lesson_type, attendance_status,grade)
Courses=# VALUES (3, 1, 9, 2, 1, '2026-05-25 10:00:00', 2, 'Весна 2026', 'Лекция', 'Присутствовал',85)
;
INSERT 0 1
Courses=# SELECT lesson_id, grade, grade_letter
Courses=# FROM lesson
[Courses=# WHERE lesson_id = (SELECT MAX(lesson_id) FROM lesson);
 lesson_id | grade | grade_letter
-----+-----+-----
          88 |      85 | Хорошо
(1 row)

Courses=# █

```

Триггер 3. Защита от изменения расписания после начала занятия

Назначение: запретить изменение даты, времени или аудитории занятия, если оно уже началось (или прошло).

```

CREATE OR REPLACE FUNCTION prevent_past_lesson_change()
RETURNS TRIGGER AS $$

```

```

BEGIN
    IF OLD.lesson_datetime < NOW() THEN
        RAISE EXCEPTION 'Ошибка: нельзя изменить прошедшее или текущее
занятие (было %)', OLD.lesson_datetime;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

DROP TRIGGER IF EXISTS tr_prevent_past_lesson_change ON lesson;
CREATE TRIGGER tr_prevent_past_lesson_change
    BEFORE UPDATE OF lesson_datetime, room_id, teacher_id ON lesson
    FOR EACH ROW
    EXECUTE FUNCTION prevent_past_lesson_change();

```

Проверка:

```

CREATE TRIGGER
[Courses=# UPDATE lesson SET room_id = 2 WHERE lesson_datetime < NOW() AND lesson_id = 1;
ERROR:  Ошибка: нельзя изменить прошедшее или текущее занятие (было 2026-05-04 09:00:00+03)
CONTEXT:  PL/pgSQL function prevent_past_lesson_change() line 4 at RAISE
Courses=# █

```

Триггер 4. Защита от переполнения аудитории

Назначение: при назначении занятия проверять, что количество студентов в группе не превышает вместимость аудитории.

```

CREATE OR REPLACE FUNCTION check_room_capacity()
RETURNS TRIGGER AS $$
DECLARE
    v_student_count INT;
    v_room_capacity INT;
BEGIN
    SELECT COUNT(*) INTO v_student_count
    FROM group_enrollment
    WHERE group_id = NEW.group_id;

    SELECT capacity INTO v_room_capacity
    FROM room
    WHERE room_id = NEW.room_id;

    IF v_student_count > v_room_capacity THEN

```

```

        RAISE EXCEPTION 'Ошибка: в аудитории % мест, а в группе %
студентов',
        v_room_capacity, v_student_count;
    END IF;

```

```

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

DROP TRIGGER IF EXISTS tr_check_room_capacity ON lesson;
CREATE TRIGGER tr_check_room_capacity
    BEFORE INSERT OR UPDATE OF room_id ON lesson
    FOR EACH ROW
    EXECUTE FUNCTION check_room_capacity();

```

Проверка:

```

[Courses=# SELECT room_id, room_number, capacity FROM room;
room_id | room_number | capacity
-----+-----+-----
1 | A-101 | 80
2 | B-202 | 30
3 | B-303 | 25
4 | B-404 | 25
5 | T-505 | 20
6 | A-202 | 80
(6 rows)

[Courses=# UPDATE room SET capacity = 1 WHERE room_id = 3;
UPDATE 1
Courses=# INSERT INTO lesson (
Courses(# individual_id, group_id, subject_id, teacher_id, room_id,
Courses(# lesson_datetime, pair_number, semester, lesson_type, attendance_status, grade
Courses(# ) VALUES (
Courses(# 13, 3, 12, 5, 3,
Courses(# '2026-05-29 10:00:00',
Courses(# 2,
Courses(# 'Весна 2026',
Courses(# 'Лекция',
Courses(# 'Присутствовал',
Courses(# 85
[Courses(# );
ERROR: Ошибка: в аудитории 1 мест, а в группе 4 студентов
CONTEXT: PL/pgSQL function check_room_capacity() line 15 at RAISE
Courses=# █

```

Триггер 5. Запрет на выставление оценки без отметки о посещении

Назначение: при добавлении оценки у студента автоматически проверяется статус посещения.

```

CREATE OR REPLACE FUNCTION check_grade_requires_attendance()
RETURNS TRIGGER AS $$
BEGIN

```

```

    IF NEW.grade IS NOT NULL AND NEW.attendance_status = 'Отсутствовал'
    THEN
        RAISE EXCEPTION 'Ошибка: нельзя выставить оценку студенту,
который отсутствовал';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

DROP TRIGGER IF EXISTS tr_check_grade_requires_attendance ON lesson;
CREATE TRIGGER tr_check_grade_requires_attendance
    BEFORE INSERT OR UPDATE OF grade, attendance_status ON lesson
    FOR EACH ROW
    EXECUTE FUNCTION check_grade_requires_attendance();

```

Проверка:

```

[Courses=# INSERT INTO lesson (individual_id, group_id, subject_id, teacher_id, room_id, lesson_datetime,]
pair_number, semester, lesson_type, attendance_status, grade) VALUES (4, 2, 10, 3, 2, '2026-05-28 12:00
:00', 3, 'Весна 2026', 'Практика', 'Отсутствовал', 85);
ERROR:  Ошибка: нельзя выставить оценку студенту, который отсутствовал
CONTEXT:  PL/pgSQL function check_grade_requires_attendance() line 4 at RAISE
Courses=# █

```

Триггер 6. Оценка только после занятия

Назначение: запретить выставлять оценку, если занятие ещё не наступило (только запланировано). Но можно выставить или изменить оценку после того, как занятие прошло

```

CREATE OR REPLACE FUNCTION check_grade_only_after_lesson()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.grade IS DISTINCT FROM OLD.grade OR (TG_OP = 'INSERT'
AND NEW.grade IS NOT NULL) THEN
        IF NEW.lesson_datetime > NOW() THEN
            RAISE EXCEPTION 'Ошибка: нельзя выставить оценку для занятия в
будущем (запланировано на %)',
                NEW.lesson_datetime;
        END IF;
    END IF;
END IF;

```



```

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS tr_check_grade_only_after_lesson ON lesson;
CREATE TRIGGER tr_check_grade_only_after_lesson
    BEFORE INSERT OR UPDATE OF grade ON lesson
    FOR EACH ROW
    EXECUTE FUNCTION check_grade_only_after_lesson();

```

Проверка:

```

CREATE TRIGGER
Courses=# INSERT INTO lesson (individual_id, group_id, subject_id, teacher_id, room_id, lesson_datetime,
    pair_number, semester, lesson_type, attendance_status, grade
Courses(# ) VALUES (3, 1, 9, 2, 1, '2026-06-01 10:00:00', 2, 'Весна 2026', 'Практика', 'Присутствовал',
85);
ERROR:  Ошибка: нельзя выставить оценку для занятия в будущем (запланировано на 2026-06-01 10:00:00+03)
CONTEXT:  PL/pgSQL function check_grade_only_after_lesson() line 5 at RAISE
Courses=# █

```

Триггер 7. Автоматическое обновление статуса студента (активен/выпущен)

Назначение: при добавлении документа об окончании автоматически менять статус студента на “Выпущен”.

```

CREATE OR REPLACE FUNCTION update_student_status_on_graduation()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE individual
    SET student_status = 'Выпущен'
    WHERE individual_id = NEW.individual_id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

DROP TRIGGER IF EXISTS tr_update_student_status_on_graduation ON
completion_document;
CREATE TRIGGER tr_update_student_status_on_graduation
    AFTER INSERT ON completion_document
    FOR EACH ROW
    EXECUTE FUNCTION update_student_status_on_graduation();

```


Проверка:

```
[Courses=# SELECT individual_id, full_name, student_status FROM individual WHERE individual_id = 1; ]
individual_id |          full_name          | student_status
```

```
-----+-----+-----
1 | Иванов Кирилл Андреевич | Активен
(1 row)
```

```
[Courses=# INSERT INTO completion_document (completion_doc_id, individual_id, program_id, issue_date, document_type)
```

```
[Courses=# VALUES (200, 1, 1, CURRENT_DATE, 'Диплом'); ]
INSERT 0 1
```

```
[Courses=# SELECT individual_id, student_status FROM individual WHERE individual_id = 1; ]
individual_id | student_status
```

```
-----+-----
1 | Выпущен
(1 row)
```

```
[Courses=# █
```

3.3 Модификация триггерной функции Части 2 практического занятия

```
CREATE OR REPLACE FUNCTION fn_check_time_punch()
RETURNS TRIGGER AS $$
DECLARE
    v_last_record RECORD;
BEGIN
    SELECT id, is_out_punch, punch_time INTO v_last_record
    FROM time_punch
    WHERE employee_id = NEW.employee_id
    ORDER BY id DESC
    LIMIT 1;

    IF v_last_record.id IS NULL THEN
        IF NEW.is_out_punch = true THEN
            RAISE EXCEPTION 'Первый заход должен быть входом';
        END IF;
        RETURN NEW;
    END IF;

    IF v_last_record.is_out_punch = false AND NEW.is_out_punch = false THEN
        RAISE EXCEPTION 'Нельзя сделать два входа подряд';
    END IF;

    IF v_last_record.is_out_punch = true AND NEW.is_out_punch = true THEN
        RAISE EXCEPTION 'Нельзя сделать два выхода подряд';
    END IF;

    IF v_last_record.is_out_punch = false AND NEW.is_out_punch = true THEN
        IF NEW.punch_time <= v_last_record.punch_time THEN
            RAISE EXCEPTION 'Время выхода не может быть раньше времени
входа';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER check_time_punch
```

```
BEFORE INSERT ON time_punch  
FOR EACH ROW  
EXECUTE FUNCTION fn_check_time_punch();
```